5주차 예습과제

📍BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

📝 교재 개념정리

1. Introduction
    논문이 다루는 분야
    해당 task에서 기존 연구 한계점
    논문의 contributions
2. Related Work
3. 제안 방법론
    Main Idea
    Contribution
4. 실험 및 결과
    Dataset
    Baseline
    결과
5. 결론 (배운점)

1. Introduction

논문이 다루는 분야

- 언어 이해(Language Understanding)를 위한 언어 표현 모델(Language Representation Model)
- 대규모 비지도 텍스트 데이터를 사용하여 심층적이고 양방향적인 언어 표현을 학습하고, 이를 다양한 자연어 처리(NLP) 태스크에 효과적으로 적용하는 방법을 제안

해당 task에서 기존 연구 한계점

- 기존의 언어 표현 모델들은 대부분 단방향(unidirectional)으로 문맥을 학습

  ex) OpenAI GPT:왼쪽에서 오른쪽으로만 정보를 처리하는 구조
  ELMo:독립적으로 학습된 왼쪽-오른쪽 및 오른쪽-왼쪽 언어 모델의 출력을 결합하는 방식

  - 이러한 단방향성이나 얕은 결합 방식은 모든 레이어에서 좌우 문맥을 동시에 활용해야 하는 작업(예: 질문 답변)에서 표현력을 제한

- 기존의 미세 조정(fine-tuning) 방식은 task-specific 아키텍처 수정이 필요하거나 사전 학습된 표현의 잠재력을 완전히 활용하지 못함

논문의 contributions

- **BERT(Bidirectional Encoder Representations from Transformers)를 제안**
  - 모든 레이어에서 좌우 문맥을 동시에 고려하는 깊고 양방향적인(deep bidirectional) 언어 표현을 사전 학습하는 모델
- **새로운 사전 학습 방법론 제시**
  - **Masked LM (MLM)**: 문장의 일부 토큰을 무작위로 마스킹하고, 양방향 문맥을 기반으로 이를 예측하는 객관식 과제(Cloze task)를 통해 양방향 정보를 효과적으로 학습하는 방법
  - **Next Sentence Prediction (NSP)**: 두 문장이 실제로 이어지는 관계인지, 아니면 무작위로 연결된 것인지를 예측하는 과제를 통해 문장 간의 관계를 학습하는 방법
- BERT는 이러한 사전 학습 모델을 가져와 추가적인 출력 레이어만으로 다양한 다운스트림 태스크에 미세 조정(fine-tuning)하는 것만으로도, 기존의 복잡한 작업별 아키텍처 없이도 11개의 NLP 태스크에서 최첨단(state-of-the-art) 성능을 달성

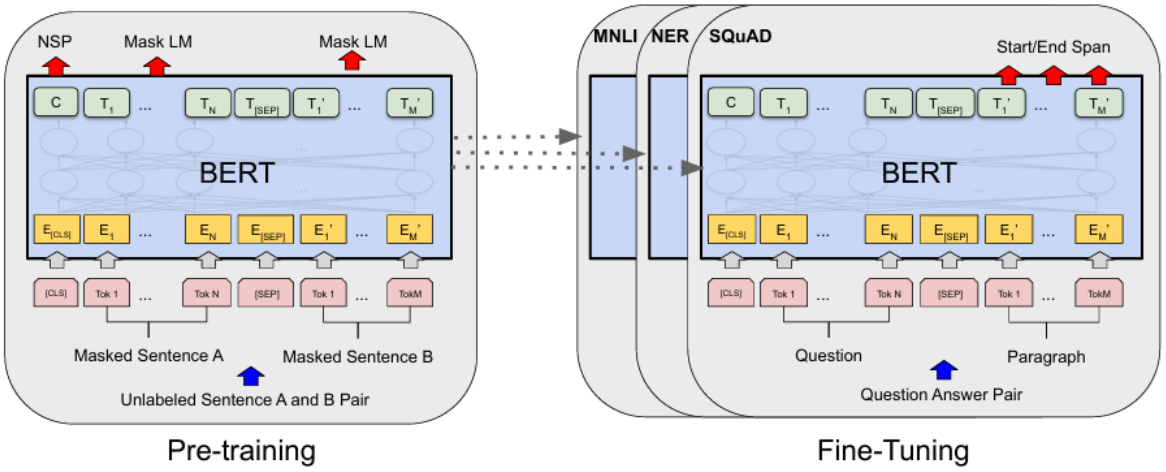
2. Related Work

- 비지도 특징 기반(Unsupervised Feature-based) 접근법
  - ELMo와 같은 모델들은 사전 학습된 언어 모델에서 추출한 특징(representation)을 다운스트림 태스크의 입력 특징으로 추가하여 사용
  - ELMo는 독립적으로 학습된 왼쪽-오른쪽(left-to-right) 및 오른쪽-왼쪽(right-to-left) 언어 모델의 출력을 결합하여 사용
  - LSTM을 사용하여 양방향 문맥으로부터 단어를 예측하는 태스크를 수행했으나, 이는 특징 기반 방식이며 깊은 양방향 모델은 X
- 비지도 미세 조정(Unsupervised Fine-tuning) 접근법
  - OpenAI GPT와 같은 모델들은 사전 학습된 파라미터를 최소한의 작업별 파라미터만 추가하여 다운스트림 태스크에서 전체 모델을 미세 조정하는 방식 사용
  - 주로 단방향 언어 모델(unidirectional language models)을 사용하여 사전 학습됨
- BERT는 미세 조정 접근법을 사용하면서도, 기존 단방향 모델들의 한계를 극복하기 위해 양방향 사전 학습을 도입

3. 제안 방법론

Main Idea

- 모든 레이어에서 깊고 양방향적인(deep bidirectional) 언어 표현을 사전 학습
  - 단일 문장 또는 문장 쌍에 대한 풍부한 문맥 정보를 모델이 이해하도록 하여, 다양한 NLP 태스크에서 더 나은 성능을 이끌어냄



1. **양방향 Transformer 인코더 아키텍처**
   - BERT는 양방향 셀프 어텐션(bidirectional self-attention)만을 사용하는 Transformer 인코더를 사용
   - 각 토큰이 시퀀스 내의 모든 다른 토큰을 주의 깊게 볼 수 있도록 하여 진정한 양방향 문맥 이해를 가능하게 함
   - BERTBASE (L=12, H=768, A=12), BERTLARGE (L=24, H=1024, A=16) 두 가지 크기 모델 구조

2. **비지도 사전 학습**
   - **Masked LM (MLM):**
     - 전통적인 언어 모델은 단방향으로만 학습되기 때문에, 깊은 양방향 모델을 학습하기 어려움

       ⇒ 입력 시퀀스의 약 15%의 토큰을 무작위로 마스킹(masking) 후, 마스킹된 토큰만을 주변 문맥을 이용하여 예측하도록 모델을 학습

- 마스킹된 토큰은 80%의 확률로 [MASK] 토큰으로 대체
  (예: "my dog is hairy" → "my dog is [MASK]")
- 10%의 확률로 무작위 단어로 대체
  (예: "my dog is hairy" → "my dog is apple")
- 10%의 확률로 원래 토큰 그대로 둠
  (예: "my dog is hairy" → "my dog is hairy")

→ 모델이 예측해야 하는 토큰이나 대체된 토큰이 무엇인지 알 수 없으므로, 각 입력 토큰에 대한 분포적이고 문맥적인 표현을 학습하도록 강제 가능

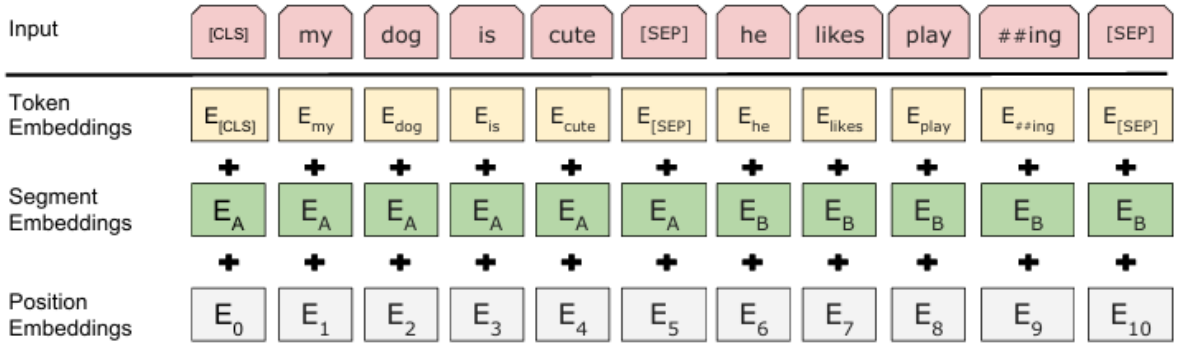
- 이때 무작위 대체는 전체 토큰의 1.5%에 불과하므로 모델의 언어 이해 능력을 유지하며 사전 학습과 미세 조정 간의 불일치(mismatch)를 줄일 수 있음

- **Next Sentence Prediction (NSP):**
  - 질의응답(QA)이나 자연어 추론(NLI)과 같이 두 문장 간의 관계 이해가 필수적인 태스크에서는 언어 모델링만으로는 부족

    ⇒ 사전 학습 데이터에서 두 문장(A, B)을 샘플링하여, B가 실제로 A의 바로 다음 문장인지(IsNext) 아니면 코퍼스 내의 무작위 문장인지(NotNext)를 예측하는 이진 분류(binary classification) 태스크를 수행

    - 50%의 확률로 B는 A의 실제 다음 문장으로, 나머지 50% 확률로 임의의 문장으로 샘플링

    → 모델이 문장 수준의 관계(예: 인과 관계, 요약, 질문-답변)를 이해하는 능력을 향상시킴



3. **입력 표현(Input Representation)**

- BERT는 단일 문장과 문장 쌍을 하나의 시퀀스로 처리할 수 있도록 설계

- 토큰화: WordPiece 임베딩(30,000 토큰 어휘)을 사용
- 특수 토큰:
  - [CLS]: 모든 시퀀스의 첫 번째 토큰으로, 분류 태스크의 최종 표현으로 사용
  - [SEP]: 문장 쌍을 구분하거나 문장 끝을 나타내는 데 사용
- 각 토큰의 최종 입력 임베딩은 토큰 임베딩(Token Embedding), 문맥(Sentence) 임베딩, 위치(Position) 임베딩의 합으로 구성됨
  - 문맥 임베딩은 토큰이 첫 번째 문장(A)에 속하는지, 두 번째 문장(B)에 속하는지를 나타냄

4. **미세 조정(Fine-tuning) 접근법**
   - 사전 학습된 BERT 모델을 가져와, 단순히 하나의 추가적인 출력 레이어를 연결
   - 그 후 레이블이 지정된 다운스트림 태스크 데이터로 모델의 모든 파라미터를 끝단까지(end-to-end) 미세 조정

     → 최소한의 작업별 파라미터만 학습하면 되므로 매우 효율적이며, 사전 학습된 깊은 양방향 표현이 다양한 태스크에 효과적으로 적용될 수 있음을 보여줌

Contribution

- **Deep Bidirectional Pre-training**
  - 기존의 단방향(left-to-right 또는 right-to-left) 언어 모델 제약을 극복
  - Transformer의 셀프 어텐션 메커니즘을 양방향으로 활용 → 모든 레이어에서 양쪽 문맥을 동시에 고려하여 모델이 단어의 의미를 더 정확하게 파악할 수 있도록 함
- **Masked LM (MLM) 사전 학습**
  - 단방향 LM이 모든 단어를 예측해야 하는 것과 달리, MLM은 일부 마스킹된 단어만을 예측하게 하여 모델이 문맥 정보를 양방향으로 고려하게 함
  - 어떤 단어를 예측해야 하는지에 대한 불확실성을 유지하면서도 양방향 문맥을 학습
- **Next Sentence Prediction (NSP) 사전 학습**
  - 문장 간의 관계(인과 관계, 요약 관계, 질문-답변 관계 등)를 이해하는 것이 중요한 다운스트림 태스크(QA, NLI 등)를 위해, 실제 이어지는 문장인지 임의의 문장인지 구분하는 NSP 태스크를 추가
- **Unified Fine-tuning Approach**

    - 사전 학습된 BERT 모델을 단일 추가 출력 레이어로 다양한 downstream 태스크에 적용, 사전 학습된 지식을 효과적으로 이전(transfer)할 수 있도록 함
- **Transformer Encoder 활용**
    - BERT는 Transformer 아키텍처를 기반으로 함
    - 양방향 셀프 어텐션(bidirectional self-attention)을 사용하는 인코더 부분을 활용

4. 실험 및 결과

💡 Introduction에서 언급한 제안 방법의 장점을 검증하기 위한 실험이 있는가

Dataset

- **Pre-training Data:**
    - BooksCorpus (800M words)와 English Wikipedia (2,500M words)를 사용
    - 긴 문맥을 학습하기 위해 문서 단위의 코퍼스를 사용
- **Downstream Task Datasets:**
    - **GLUE Benchmark:** MNLI, QQP, QNLI, SST-2, CoLA, STS-B, MRPC, RTE, WNLI (WNLI는 제외됨) 등 9개의 태스크를 포함
    - **SQuAD v1.1:** 100k+ 질문/답변 쌍으로 구성된 기계 독해 데이터셋
    - **SQuAD v2.0:** SQuAD v1.1에 "답변이 없는" 경우를 추가하여 현실성을 높인 데이터셋
    - **SWAG:** 113k 문장-연속 쌍으로 구성된 상식 추론 데이터셋
    - **CoNLL-2003 NER:** Named Entity Recognition (NER) 태스크를 위한 데이터셋

Baseline

- BERT의 성능을 다양한 최신(State-of-the-Art) 모델들과 비교
    - Feature-based Models: ELMo , BiLSTM+ELMo+Attn
    - Fine-tuning Models: OpenAI GPT
    - Task-specific Models: BiDAF+ELMo, R.M. Reader, QANet, U-Net, ESIM+GloVe, ESIM+ELMo 등 각 태스크별로 당시 최고 성능을 보였던 모델들.

- Human Performance

결과

- BERT는 11개의 NLP 태스크에서 모두 이전 SOTA를 경신
  - 특히 GLUE 벤치마크에서 평균 4.5%~7.0%의 높은 정확도 향상
  - MNLI 태스크에서는 4.6%의 절대적 정확도 향상
- **SQuAD:** SQuAD v1.1에서는 단일 모델로도 기존 앙상블 시스템보다 높은 F1 점수를 달성, SQuAD v2.0에서는 이전 최고 시스템 대비 +5.1 F1 향상
- **SWAG:** BERTLARGE는 ESIM+ELMo 대비 +27.1%, OpenAI GPT 대비 8.3%의 정확도 향상

- **Ablation Study**
  - Effect of Pre-training Tasks (사전 학습 태스크의 영향):
    - NSP 태스크 없이 MLM만 사용한 모델(No NSP)과, Left-to-Right LM으로 학습하고 NSP가 없는 모델(LTR & No NSP)을 비교
    - NSP 태스크 제거 시 QNLI, MNLI, SQuAD에서 성능이 유의미하게 하락
    - LTR 모델은 MLM 모델보다 모든 태스크에서 성능이 낮았음

      → 양방향성의 중요성 보여줌
  - Effect of Model Size (모델 크기의 영향):
    - 다양한 레이어 수, 히든 사이즈, 헤드 수를 가진 BERT 모델들을 실험

      → 모델 크기가 증가함에 따라 성능이 꾸준히 향상됨
    - 대규모 모델이 작은 데이터셋에서도 이점을 가질 수 있음
  - Feature-based Approach with BERT:
    - BERT를 fine-tuning 방식뿐만 아니라 feature-based 방식으로도 NER 태스크에 적용

      → BERT의 사전 학습된 표현이 다양한 방식(fine-tuning, feature extraction)으로 활용될 수 있음을 보여줌

5. 결론 (배운점)

💡 연구의 의의 및 한계점, 본인이 생각하는 좋았던/아쉬웠던 점 (배운점)

- 마스크드 언어 모델(MLM)을 통해 깊고 양방향적인(deeply bidirectional) 언어 표현을 학습함으로써, 이전 단방향 모델들이 가졌던 한계를 극복하고 문맥을 훨씬 더 정확하고 풍부하게 이해할 수 있게 됨
- 대규모의 레이블 없는 데이터로 사전 학습한 후, 적은 양의 레이블 데이터로 미세 조정(fine-tuning)하는 접근 방식을 통해 복잡한 태스크별 구조 설계 없이도 다양한 NLP 문제에서 최신(state-of-the-art) 성능을 달성 → 후속 LLM 연구의 기초가 됨
- 한계점
  - 높은 사전 학습 비용: BERT 모델, 특히 BERTLARGE는 방대한 양의 데이터와 컴퓨팅 자원을 필요로 하여 상당한 학습 비용이 발생
  - [MASK] 토큰 불일치: 사전 학습 시 사용되는 [MASK] 토큰이 미세 조정 단계에서는 나타나지 않아 미묘한 불일치가 발생할 수 있음

    → 완화하기 위한 전략(무작위 단어 대체, 원본 유지)이 사용되었지만 추가적인 방법이 필요할 수 있음
  - 문장 및 단락 길이 제한 (Sequence Length Limitations):
    - BERT는 트랜스포머 아키텍처를 기반으로 하며, 트랜스포머의 어텐션 메커니즘은 시퀀스 길이에 대해 O(n^2)의 계산 복잡도를 가짐
    - 따라서 BERT는 일반적으로 최대 512 토큰의 입력 시퀀스 길이로 제한됨

      → 긴 문서나 대화와 같이 더 긴 문맥이 필요한 작업에는 직접적으로 적용하기 어렵다는 한계 존재

🧐

- 깊고 양방향적인 언어 표현 사전 학습 방식을 통해 단순히 왼쪽에서 오른쪽 등 단방향이 아닌 양방향을 동시에 고려하여 문맥을 더 정확하게 이해할 수 있다는 사실이 인상깊었다. 인간의 인지 과정과 더 가까워진 접근 방식이라고 생각한다.
- 사전 학습과 미세 조정이라는 두 가지 접근 방식의 효과가 매우 강력하고, 따라서 태스크별 복잡한 구조 설계 없이도 뛰어난 성능을 낼 수 있다는 사실을 배울 수 있었다.

교재 개념정리

🔖<(파이토치 트랜스포머를 활용한) 자연어 처리와 컴퓨터 비전 심층학습>

- BERT (406~416p)

BERT

- **BERT(Bidirectional Encoder Representations from Transformers)**
  - 2018 구글에서 발표한 언어 모델
  - 트랜스포머 기반 양방향 인코더를 사용하는 자연어 처리 모델

- **양방향(Bidirectional)** 인코더
  - 입력 시퀀스를 양쪽 방향에서 처리 → 이전, 이후 단어를 모두 참조하면서 단어의 의미와 문맥 파악 가능
    - 기존 언어모델은 문장을 좌에서 우로 순차 학습해 이전 단어만 고려
- BERT의 특징
  - 양방향 학습을 통해 더 정확한 문맥 파악 가능
  - 대규모 데이트를 사용한 사전 학습되어 있어 전이 학습에 주로 활용
  - 일부 또는 전체를 다른 작업에서 재사용해 적은 양의 데이터로도 높은 정확도 달성 가능

- BERT는 트랜스포머의 인코더를 기반으로 함

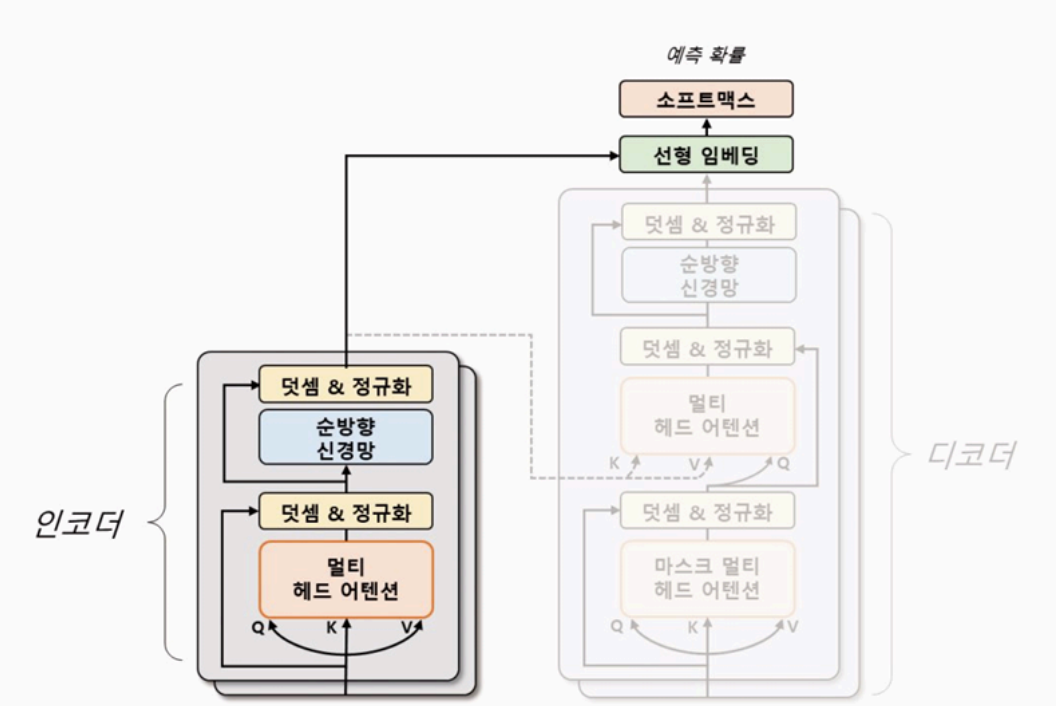


그림 7.13 트랜스포머와 BERT 모델 비교

- 트랜스포머 인코더 모듈만을 사용해 입력 문장 처리
  - 입력 문장의 단어 임베딩해 각 단어의 의미를 벡터화
  - 이후 이를 순차적으로 처리하여 문장 전체의 의미를 추출

사전 학습 방법

1. **마스킹된 언어 모델링(Masked Language Modeling, MLM)**

- 입력 문장에서 임의로 일부 단어를 마스킹하고 해당 단어를 예측하는 방식
  - ex) “I’m learning PyTorch”

    ‘learning’을 마스킹 → ‘I’m [MASK] Pytorch’

    BERT는 양방향 문맥 정보를 참고해 [MASK]에 들어갈 단어를 예측
- 양방향으로 문장을 학습해 문맥 정보를 모델링하므로 입력 문장에서 누락된 단어를 추론하는 능력 갖게 됨 → 문장 전체 의미 이해 능력 상승

2. **다음 문장 예측(Next Sentence Prediction, NSP)**

- 두 개의 문장이 주어졌을 때, 두 번째 문장이 첫 번째 문장의 다음에 오는 문장인지 여부를 판단하는 작업

- ex) ‘I’m learning PyTorch’, ‘PyTorch is a machine learning library’

  → 두 문장이 연속적인 관계인지 아닌지를 예측

- 예측 과정에서 두 문장 간의 관계를 학습하고, 문장 간의 의미적인 유사성을 파악

- BERT 모델은 입력 문장에 특수 토큰을 추가해 정보를 제공
  - **[CLS] 토큰**
    - 입력 문장의 시작 부분에 추가
    - 입력 문장이 어떤 유형의 문장인지 미리 파악 가능
  - **[SEP] 토큰**
    - 입력 문장 내에서 두 개 이상의 문장을 구분하기 위해 사용
    - 입력 문장을 두 개의 독립적인 문장으로 인식하고, 각 문장에 대한 정보를 파악 가능
  - **[MASK] 토큰**
    - 입력 문장 내에서 임의로 선택된 단어를 가리키는 토큰
    - 주어진 문장에서 일부 단어를 가린 후 학습과 예측에 활용
- BERT 모델의 입력 시퀀스는 **[CLS] 문장-1 [SEP]문장-2 [MASK] [SEP]** 등의 구조 가질 수 있음

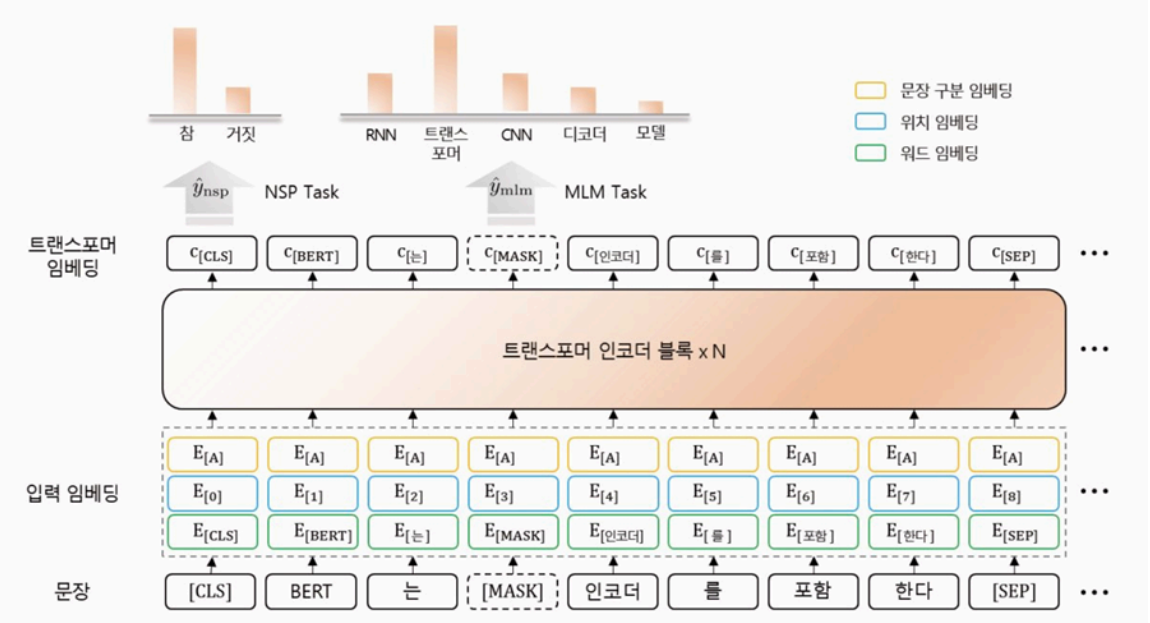


그림 7.14 BERT 모델의 입력 임베딩과 손실 함수

    - 문장 앞에 [CLS] 토큰, 문장 마지막에 [SEP] 토큰을 추가
    - 텍스트 토큰 중 15%에 해당하는 단어를 대상으로 마스킹 수행
        - 이 중 80%는 [MASK] 토큰으로 대체, 10%는 어휘 사전에 존재하는 무작위 단어로 변경, 나머지 10%는 실제 토큰 그대로 사용

- BERT는 사전 학습 수행 후 미세 조정 기법을 통해 다양한 자연어 처리 작업에 적용 가능
    - 문장 분류, 감성 분석, 질문 응답, 기계 번역 등
    - 미세 조정 과정에서는 해당 작업에 맞는 계층을 추가하고 손실 함수를 정의해 학습
    - 분류 문제에서는 [CLS] 토큰 벡터를, 각 토큰마다 예측이 필요한 경우에는 모든 토큰 벡터를 사용

모델 실습

- 허깅 페이스 트랜스포머스 라이브러리의 BERT 모델과 네이버 영화 리뷰 감정 분석 데이터세트를 활용해 분류 모델을 학습

```python
#네이버 영화 리뷰 데이터 불러오기
import numpy as np
import pandas as pd
from Korpora import Korpora

corpus = Korpora.load("nsmc")
df=pd.DataFrame(corpus.test).sample(20000, random_state=42)
train, valid, test = np.split(
    df.sample(frac=1, random_state=42), [int(0.6*len(df)),int(0.8*len(df))]
)

print(train.head(5).to_markdown())
print(f"Training Data Size : {len(train)}")
print(f"Validation Data Size : {len(valid)}")
print(f"Testing Data Size : {len(test)}")
```

```
|       | text                                                    |   label |
|------:|:------------------------------------------------------|--------:|
| 26891 | 역시 코믹액션은 성룡, 홍금보, 원표 삼인방이 최고지!!     |       1 |
| 25024 | 점수 후하게 줘야것네 별 반개~                            |       0 |
| 11666 | 오랜만에 느낄수 있는 [감독] 구타욕구.                     |       0 |
| 40303 | 본지는 좀 됬지만 극장서 돈주고 본게 아직까지 아까운 영화 |       0 |
| 18010 | 징키스칸이란 소재를 가지고 이것밖에 못만드냐              |       0 |
Training Data Size : 12000
Validation Data Size : 4000
Testing Data Size : 4000
```

- 20,000개의 데이터세트를 6:2:2로 분리해 학습, 검증, 테스트 데이터로 활용

```python
#BERT 입력 텐서 생성
import torch
from transformers import BertTokenizer
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler

def make_dataset(data, tokenizer, device):
    tokenized = tokenizer(
        text=data.text.tolist(),
        padding="longest",
        truncation=True,
        return_tensors="pt"
    )
    input_ids = tokenized["input_ids"].to(device)
    attention_mask = tokenized["attention_mask"].to(device)
    labels = torch.tensor(data.label.values, dtype=torch.long).to(device)
    return TensorDataset(input_ids, attention_mask, labels)

def get_datalodader(dataset, sampler, batch_size):
    data_sampler = sampler(dataset)
    dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)
    return dataloader

epochs = 5
batch_size = 32
device = "cuda" if torch.cuda.is_available() else "cpu"
tokenizer = BertTokenizer.from_pretrained(
    pretrained_model_name_or_path="bert-base-multilingual-cased",
    do_lower_case=False
)
```

```python
train_dataset = make_dataset(train, tokenizer, device)
train_dataloader = get_datalodader(train_dataset, RandomSampler, batch_size)

valid_dataset = make_dataset(valid, tokenizer, device)
valid_dataloader = get_datalodader(valid_dataset, SequentialSampler, batch_size)

test_dataset = make_dataset(test, tokenizer, device)
test_dataloader = get_datalodader(test_dataset, SequentialSampler, batch_size)

print(train_dataset[0])
```

```
(tensor([   101,  58466,   9812, 118956, 119122,  59095,  10892,   9434, 118888,
           117,   9992,  40032,  30005,    117,   9612,  37824,   9410,  12030,
         42337,  10739,  83491,  12508,    106,    106,    102,      0,      0,
             0,      0,      0,      0,      0,      0,      0,      0,      0,
             0,      0,      0,      0,      0,      0,      0,      0,      0,
             0,      0,      0,      0,      0,      0,      0,      0,      0,
             0,      0,      0,      0,      0,      0,      0,      0,      0,
             0,      0,      0,      0,      0,      0,      0,      0,      0,
             0,      0,      0,      0,      0,      0,      0,      0,      0,
             0,      0,      0,      0,      0,      0,      0,      0,      0,
             0,      0,      0,      0,      0,      0,      0,      0,      0,
             0,      0,      0,      0,      0,      0,      0,      0,      0,
             0,      0,      0,      0,      0,      0,      0],
      device='cuda:0'), tensor([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0], device='cuda:0'), tensor(1, device='cuda:0'))
```

- BERT 토크나이저 클래스로 데이터를 전처리
  - 사전 학습된 BERT 모델 bert-base-multilingual-cased
    - 다중 언어 지원, 대소문자 유지
    - 소문자 유지(do_lower_case)를 False로 지정해 소문자로 변환하지 않게 함(Apple과 apple을 같은 단어로 인식하게 됨)
- 토크나이저를 make_dataset 함수에 전달해 텐서 데이터세트를 반환 → 출력 결과 텍스트는 숫자 ID와 어텐션 마스크로 변경, 레이블이 텐서 형식으로 변환됨
- 이후 데이터로더를 적용
  - get_dataloader 함수: 샘플러 클래스를 활용해 데이터를 목적에 따라 샘플링
  - 무작위 샘플러(RandomSampler) 클래스: 데이터를 무작위로 샘플링 → 학습에 적용
  - 시퀀셜 샘플러(SeqentialSampler) 클래스: 데이터를 고정된 순서대로 반환 → 검증 및 평가 배치에 적용

```python
#BERT 모델 선언
from torch import optim
from transformers import BertForSequenceClassification

model = BertForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="bert-base-multilingual-cased",
    num_labels=2
).to(device)
optimizer = optim.AdamW(model.parameters(), lr=1e-5, eps=1e-8)
```

- BERT 모델을 BERT 문장 분류 모델(BertForSequenceClassification) 클래스로 불러옴
- 활성화 함수: AdamW
  - Adam 최적화 함수에 가중치 감쇠 추가한 변형된 경사 하강법 알고리즘
  - 엡실론(eps) 매개변수를 분모에 더해 학습률을 0으로 나누는 것을 방지
  - 갱신되는 모든 매개변수의 학습률을 조절하는 적응형 활성화 함수이므로 모델의 빠른 수렴, 안정적 학습 가능

👩‍💻

```
bert
L embeddings
¦ L word_embeddings
¦ L position_embeddings
¦ L token_type_embeddings
¦ L LayerNorm
¦ L dropout
L encoder
¦ L layer
¦ ¦ L 0
¦ ¦ L 1
¦ ¦ L 2
¦ ¦ L 3
¦ ¦ L 4
¦ ¦ L 5
¦ ¦ L 6
¦ ¦ L 7
¦ ¦ L 8
¦ ¦ L 9
¦ ¦ L 10
¦ ¦ L 11
L pooler
¦ L dense
¦ L activation
dropout
classifier
```

- BERT 모델의 입력 임베딩
  - 토큰 임베딩(word_embeddings), 위치 임베딩(position_embeddings), 문장 구분 임베딩(token_type_embeddings)으로 구성
  - 서로 다른 정보 담고 있으며, 이를 결합해 입력 시퀀스에 대한 임베딩 벡터를 생성
  - 생성된 임베딩 벡터는 계층 정규화와 드롭아웃 수행
- 트랜스포머 인코더 블록
  - 총 12개를 사용
  - 각 블록은 입력 시퀀스를 순차적으로 처리해 새로운 임베딩을 생성
  - 이 과정에서 셀프 어텐션 및 순방향 신경망을 사용해 입력 시퀀스의 다양한 관계를 학습
- 풀러(pooler)

    - [CLS] 토큰 벡터를 한 번 더 비선형 변환을 수행하기 위해 선형 변환과 비선형 변환인 Tanh 함수 사용
- 드롭아웃 적용 → 모델의 과대적합 방지
- 분류기(classifier)
    - BERT 모델에서 수행하는 작업, [CLS] 토큰 벡터를 활용해 결과를 예측

- BERT 모델 학습 및 평가 결과

```
Epoch 1: Train Loss: 0.4344 Val Loss: 0.4126 Val Accuracy: 0.8090
Saved the model weights
Epoch 2: Train Loss: 0.3482 Val Loss: 0.4382 Val Accuracy: 0.8153
Epoch 3: Train Loss: 0.2764 Val Loss: 0.4505 Val Accuracy: 0.8153
Epoch 4: Train Loss: 0.2040 Val Loss: 0.4823 Val Accuracy: 0.8227
Epoch 5: Train Loss: 0.1646 Val Loss: 0.5547 Val Accuracy: 0.8147
```

```
Test Loss : 0.4179
Test Accuracy : 0.8070
```